

EE488 HW2 Report

학번 비공개 Sim Hyeonseong

To select a model between fully-connected and convolutional to run easily, and to make 6 possible combinations of NORMAL_NUM and LATENT_DIM easily, I modified some codes in part "2. Hyperparameters" like below. Also, I fixed random seed as 0 adding the code 'torch.manual_seed(0)' to generate the same results for every run.

```
1 torch.manual_seed(0) # Fix random seed
2
3 MODEL = 0 # Select a model between fully-connected and convolutional to run
4
5 # MODEL | Type of AutoEncoder
6 # 0 | Fully-connected
7 # 1 | Convolutional
8
9 COMBINATION = 0 # 6 possible combination of LATENT_DIM and NORMAL_NUM(ANOMALY_NUM) | from 0 to 5
10
11 # COMBINATION | NORMAL_NUM | ANOMALY_NUM | LATENT_DIM
12 # 0 | 1 | 0,2,3,4,5,6,7,8,9 | 4
13 # 1 | 1 | 0,2,3,4,5,6,7,8,9 | 64
14 # 2 | 1 | 0,2,3,4,5,6,7,8,9 | 128
15 # 3 | 1,2 | 0,3,4,5,6,7,8,9 | 4
16 # 4 | 1,2 | 0,3,4,5,6,7,8,9 | 64
17 # 5 | 1,2 | 0,3,4,5,6,7,8,9 | 128
18
19 NORMAL_NUM = [[1], [1,2]][COMBINATION // 3] # (list) Data of digit "1", or digits "1, 2" will be used as normal data
20 ANOMALY_NUM = [i for i in range(10) if i not in NORMAL_NUM] # (list) Data of digits not in NORMAL_NUM will be used as anomalous data
21
22 LATENT_DIM = [4, 64, 128][COMBINATION % 3] # latent dimension
23
24 EPOCHS = 20 # Number of epochs to train
25 BATCH = 32 # Minibatch size

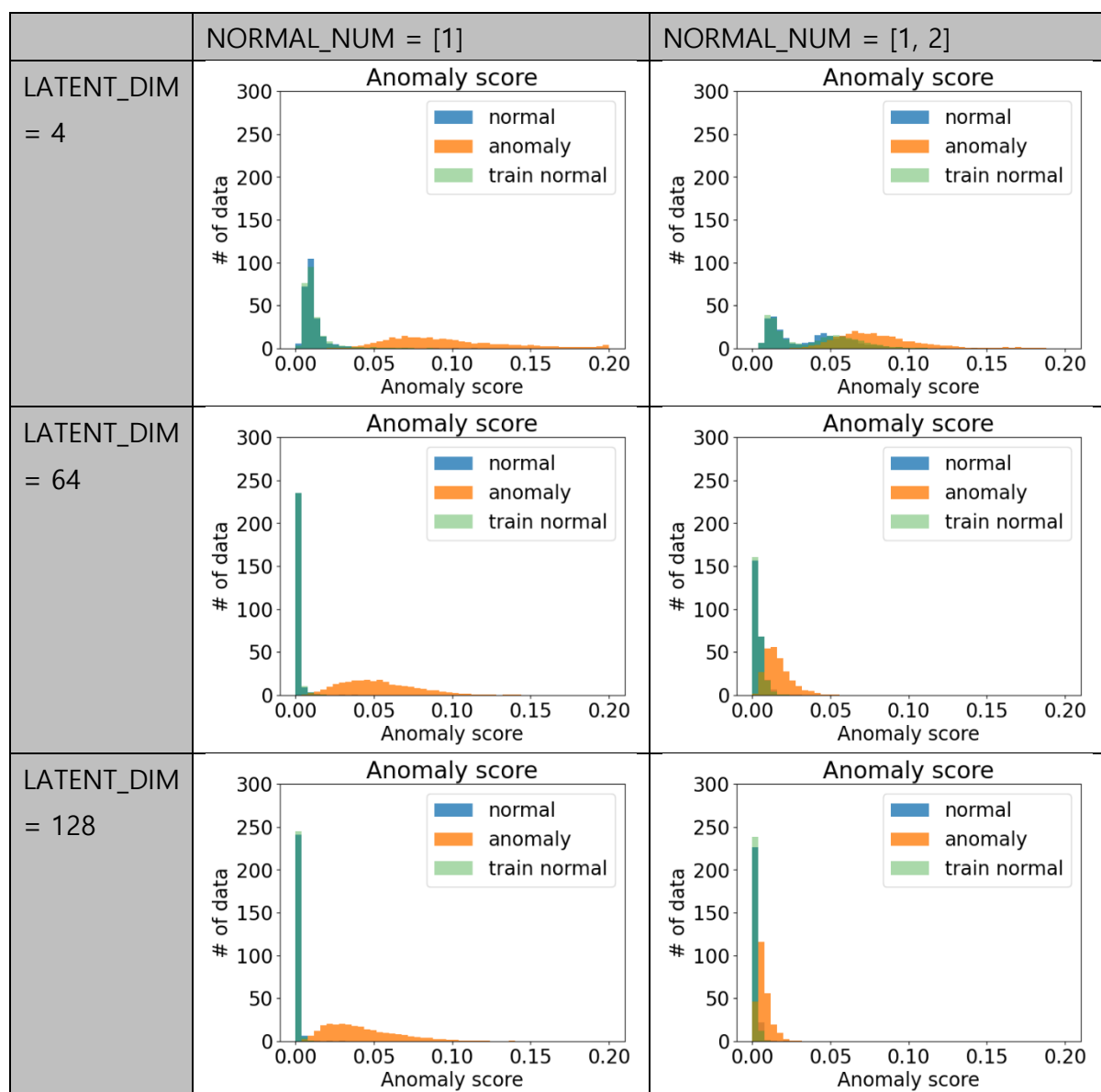
```

```
1 ## Create autoencoder object
2 if MODEL == 0:
3     model = AutoEncoder(LATENT_DIM).to(device)
4 else:
5     model = ConvAutoEncoder(LATENT_DIM).to(device)
6
7 summary(model, input_size=(BATCH, 1, 28, 28)) # summary(model, input_size)
```

For the variable MODEL, the model to train will be fully-connected autoencoder when MODEL is 0, and it will be convolutional autoencoder otherwise. The variable COMBINATION can vary from 0 to 5. If COMBINATION = 0, 1, or 2, NORMAL_NUM = [1] and ANOMALY_NUM = [0, 2, 3, 4, 5, 6, 7, 8, 9]. Otherwise, NORMAL_NUM = [1, 2] and ANOMALY_NUM = [0, 3, 4, 5, 6, 7, 8, 9]. Also, if the remainder of COMBINATION divided by 3 is 0, then LATENT_DIM = 4, and if the remainder is 1, then LATENT_DIM = 64. Otherwise, LATENT_DIM = 128.

[Problem 1]

[A] I ran the whole code 6 times with fully-connected autoencoder model.



The figures above are 6 histograms from each run.

First, let's compare the difference between latent dimensions. As the latent dimension increases, the histogram gradually moves to the left. More specifically, the anomaly score of each sample is getting smaller when the latent dimension is higher. The overlapping parts in histogram of latent dimension 4 between normal and anomaly is larger than those of latent dimension 64 or 128, especially when NORMAL_NUM = [1, 2]. In other words, it is observed that distribution with latent dimension 4 between normal and abnormal data tends to be less clearly distinguished, especially when NORMAL_NUM = [1, 2].

Comparing the difference between NORMAL_NUM = [1] and NORMAL_NUM = [1, 2], the overlapping parts in histogram of [1, 2] between normal and anomaly is much higher than those of [1]. Plus, error distribution of normal data is wider in [1, 2].

[B] Lower latent dimension (e.g. 4) results in higher anomaly scores since there is more information loss of input image so the performance of reconstructing complex image might be bad. Higher latent dimension (e.g. 128) preserves more information so that the model can extract enough features from input images to reconstruct them. However, if we construct a model with too high latent dimension, then it might reconstruct not only normal but also anomaly well. So, the anomaly score histograms between normal and anomaly are getting closer, and decision boundary between them becomes ambiguous.

Additionally, when NORMAL_NUM = [1], the normal data is composed of a single class, so that the model learns more specific pattern from those similar data. Thus, it becomes an expert in reconstructing images of 1, and histograms between normal and anomaly are separated well. On the other hand, model training with multiple classes [1, 2] must learn more diverse patterns, and it needs more generalization on training data. As a result, error distribution becomes widened within normal data, and overlapping parts of histograms between normal and anomaly becomes larger compared to training with [1].

[Problem 2]

[A]

```
1 ## Define the autoencoder class
2 class ConvAutoEncoder(nn.Module):
3     def __init__(self, encoding_dim):
4         super(ConvAutoEncoder, self).__init__()
5         self.encoding_dim = encoding_dim
6         self.encoder = nn.Sequential(
7             nn.Conv2d(1, 2, kernel_size = 3, stride=2, padding=1), # b, 2, 14, 14
8             nn.ReLU(),
9
10            ## Fill required layers here
11            nn.Conv2d(2, 4, kernel_size = 3, stride=2, padding=1), # b, 4, 7, 7
12            nn.ReLU(),
13            nn.Conv2d(4, 8, kernel_size = 3, stride=2, padding=1), # b, 8, 4, 4
14            nn.ReLU(),
15            nn.Conv2d(8, 16, kernel_size = 3, stride=2, padding=1), # b, 16, 2, 2
16            nn.ReLU(),
17            nn.Flatten(), # b, 64
18
19            nn.Linear(16 * 2 * 2, encoding_dim),
20            nn.ReLU(),
21            nn.Linear(encoding_dim, encoding_dim),
22            nn.ReLU()
23        )
```

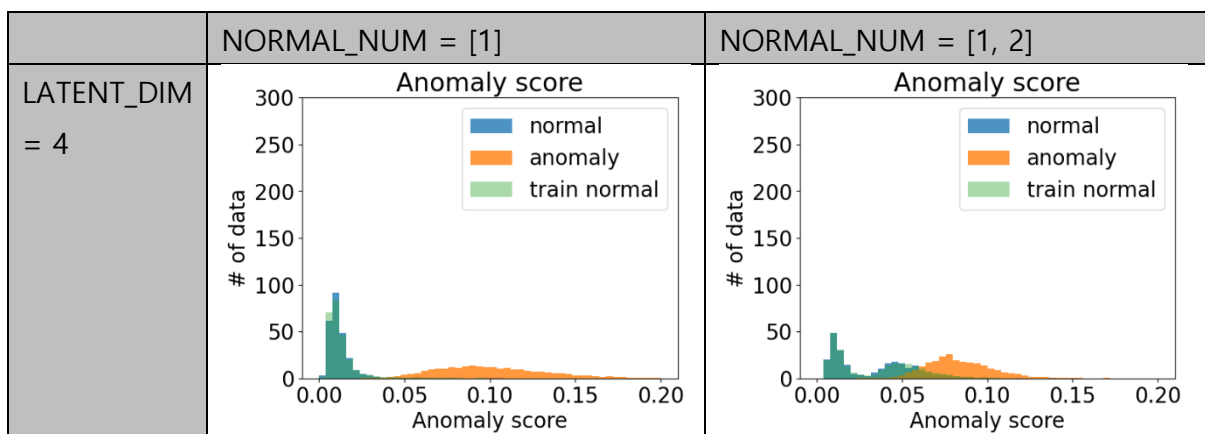
```

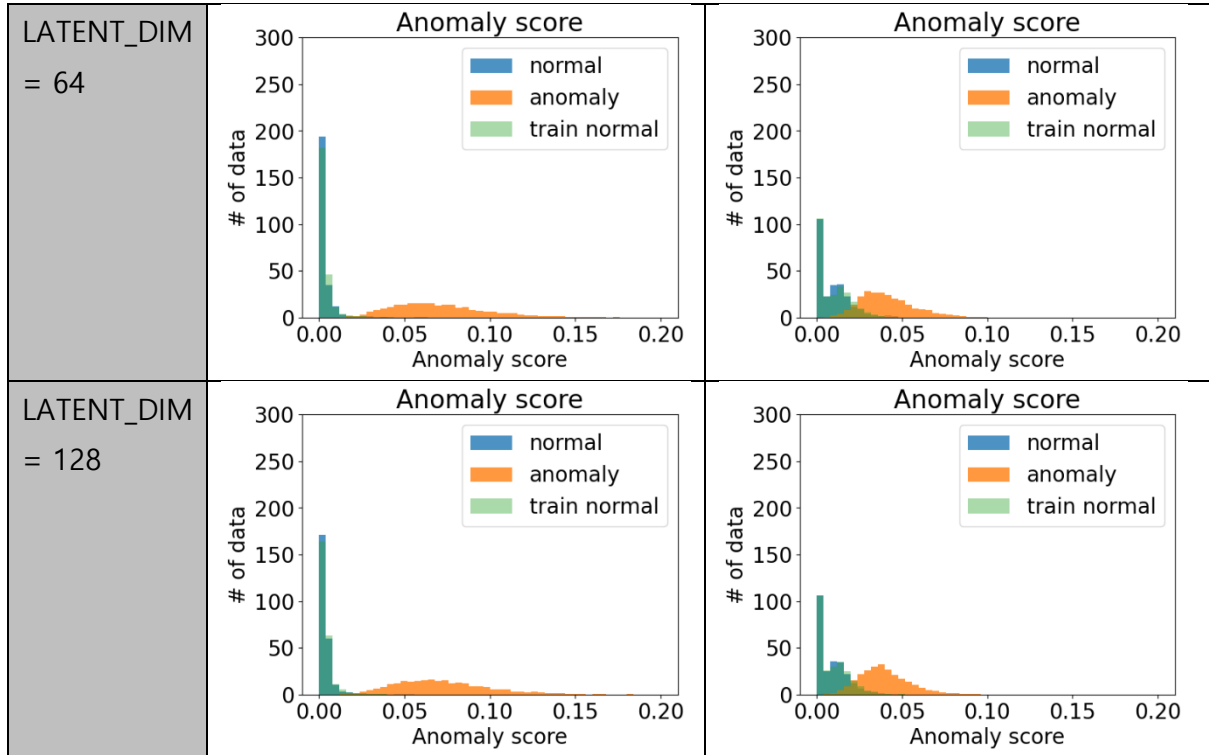
24     self.decoder = nn.Sequential(
25         nn.Linear(encoding_dim, encoding_dim),
26         nn.ReLU(),
27         nn.Linear(encoding_dim, 16*2*2),
28         nn.ReLU(),
29         nn.Unflatten(1, (16, 2, 2)),
30
31         ## Fill required layers here
32         nn.ConvTranspose2d(16, 16, kernel_size=3, stride=2, padding=1, output_padding=1), # b, 16, 4, 4
33         nn.ReLU(),
34         nn.ConvTranspose2d(16, 8, kernel_size=3, stride=2, padding=1), # b, 8, 7, 7
35         nn.ReLU(),
36         nn.ConvTranspose2d(8, 4, kernel_size=3, stride=2, padding=1, output_padding=1), # b, 4, 14, 14
37         nn.ReLU(),
38         nn.ConvTranspose2d(4, 2, kernel_size=3, stride=2, padding=1, output_padding=1), # b, 2, 28, 28
39         nn.ReLU(),
40         nn.Conv2d(2, 1, kernel_size=3, padding=1), # b, 1, 28, 28
41         nn.Sigmoid()
42     )
43
44     self.initialize_weights() # Call Weight Initialization Function
45
46     def initialize_weights(self): # Weight Initialization Function
47         for m in self.modules():
48             if isinstance(m, nn.Conv2d) or isinstance(m, nn.ConvTranspose2d) or isinstance(m, nn.Linear):
49                 nn.init.kaiming_normal_(m.weight, mode='fan_in', nonlinearity='relu') # He Initialization
50                 if hasattr(m, 'bias') and m.bias is not None:
51                     nn.init.zeros_(m.bias)
52
53     def forward(self, x):
54         #out = x.reshape(x.size(0), -1) # (B,1,28,28) -> (B,784)
55         z = self.encoder(x)
56         out = self.decoder(z)
57         out = out.view(x.size()) # (B,784) -> (B,1,28,28)
58         return out

```

I constructed various layers in ConvAutoEncoder class like above to satisfy all instructions in the assignment pdf file. I also added function, initialize_weights(self), in the class to generate proper results since if initial weights are not appropriate for the model, then change of loss might stop somewhere, and reconstructed images might be strange.

[B] In this time, I ran the whole code 6 times with convolutional autoencoder model.





The overall appearance of error distributions above is almost the same as that from fully-connected autoencoder. So, in this part, I will focus on describing differences of histograms between fully-connected and convolutional.

Overall shifts toward left of the histograms for higher latent dimension in convolutional model is much weaker than in fully-connected one. The histograms of both normal and anomaly are in almost the same positions regardless of latent dimensions. While the anomaly scores of normal data concentrate near 0, those of anomalies tend to spread wider at the positions somewhat far from those of normal data. Thus, ratio of overlapping parts in histograms between normal and anomaly is relatively smaller in convolutional model. Since the distributions of normal and anomaly are more clearly separated, it is easier to select decision boundary.

[C] A fully-connected autoencoder learns flattened data ignoring spatial structure from them. On the other hand, a convolutional autoencoder learns original image data maintaining the spatial structure of them. It means that convolutional autoencoder specializes in encoding and decoding data of image type, which has spatial features such as edge, shape, or boundary. Therefore, the convolutional autoencoder can extract more meaningful local patterns, and capture differences better between normal and anomaly. Also, it has convolutional layers which use parameter sharing and local connectivity, so the number of parameters in the layers is small and it is unlikely that the model overfits. As a result, reconstruction errors of normal data are low and those of abnormal data are high

so that error distributions between them are quite clearly separated and overlapping part is small. Unlike convolutional autoencoder, it is highly likely that the error distribution of anomaly approaches to that of normal in fully-connected autoencoder because it reconstructs both normal and abnormal input images with similar qualities. Since it treats all input pixels equally, its latent space is less constructive. In other words, since it tries to comprehend each pixel one by one, not some meaningful local chunks from the image, the latent vectors of anomalies may be close to the latent vectors distribution of normal data.